

INCOMPLETENESS FOR STABLY COMPUTABLE THEORIES

YASHA SAVELYEV

ABSTRACT. Using the theory of Turing machines, we give a set theoretic reframing of Gödel's first and second incompleteness theorems that applies even to some arithmetically undefinable theories. The usual Hilbert–Bernays provability conditions, the “diagonal lemma”, and even usage of numerals are absent, replaced by a more direct diagonalization argument, from first principles, based in set theory and in a sense analogous to Cantor's original argument. To this end, we partially categorify the theory Gödel encodings, which might be of independent interest. There are various upshots. We show that Gödel sentence, is computably constructive (at least at the level of generality here). Moreover, our set theory reframed version of the second incompleteness theorem is in some ways stronger than the classical analogue. In an appendix, we formalize a version of the partly physical argument of Roger Penrose, using this reframed second incompleteness theorem.

1. INTRODUCTION

The classical Gödel incompleteness theorems are formulated for definable arithmetic theories. In fact, the original incompleteness theorem asked for a computably enumerable theory of arithmetic, but the latter can be generalized in various ways to Σ_n theories. See for instance [?]. One of the goals here is to extend this to certain undefinable theories of arithmetic.

We will work from first principles, and totally within the meta-theory *ZFC*, that is all theorems are theorems of *ZFC* (once totally formalized). Our setup will be language agnostic, we work with a general $\mathcal{L}$ -theory, interpreting arithmetic but the interpretation need not be arithmetically definable. Partly for this reason various standard tools like Hilbert–Bernays provability conditions, the “diagonal lemma” and even numerals will not appear, as translating them to our context is difficult.¹

Instead, the diagonalization argument that we use is more direct, elementary², and highly analogous to the diagonalization argument of Cantor himself, as well as to Turing's original work on the halting problem. To this end we also partially categorify the theory of Gödel encodings, essentially making explicit what is always done implicitly. This might be of independent interest.

The primary motivation and many of the choices we make, most notably the choice to work with stably c.e. theories (see following section), are based around the problem of formalizing a certain version of an argument of Roger Penrose on potential non-computability in physics. The reader may see Appendix A, however many terms there are defined throughout the paper.

1.1. Definition of stably c.e. theories and statements of results. Let $\mathcal{L}$ be a first order language, in particular a formal language in a countable alphabet including the symbols of first order logic, with syntax of first order logic, such that elements of $\mathcal{L}$ are sentences (in the logical sense).

We now quickly introduce stable c.e. theories. In what follows, a map is always a partial map unless specified to be total. Suppose we are given a map

$$M : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\},$$

for $\{\pm\}$ denoting a set with two elements $+$, $-$.

¹Probably not impossible.

²If we chose to work with standard c.e. sets as opposed to stably c.e. sets the argument would be even more elementary.

Definition 1.1. We say that $\alpha \in \mathcal{L}$ is **M -stable** if there is an m with $M(m) = (\alpha, +)$ s.t. there is no $n > m$ with $M(n) = (\alpha, -)$. Let $M^s \subset \mathcal{L}$ denote the set of M -stable α , called **the stabilization of M** .

Remark 1.2. For an informal motivation of how such an M may appear in practice consider the following. With $\mathbb{N}$ playing the role of time, M might be a mathematician producing sentences of arithmetic that it believes to be true, at each moment $n \in \mathbb{N}$. But M is also allowed to correct itself in the following sense.

- $M(n) = (\alpha, +)$, only if at the moment n M decides that α is true.
- $M(m) = (\alpha, -)$, only if at the moment m , M no longer asserts that α is true, either because at this moment M is no longer able to decide α , or because it has decided it to be false.

Definition 1.3. A subset $S \subset \mathcal{L}$, is called **stably computably enumerable** or **stably c.e.**, if there is a computable map (see Definition 2.9) $T : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$ so that $S = T^s$. In this case we also say that T **stably enumerates** S . We say that $T \in \mathcal{T}$ **stably computes** $M : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$, if it computes some $N : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$, s.t. $M^s = N^s$.

It is fairly immediate that a stably c.e. S is Σ_2 definable. The converse is also true, every Σ_2 definable set $S \subset \mathcal{L}$ is stably c.e.. To prove this we may build on Example 3.3, to construct an oracle and then use the theorems of Post, (see [14]), relating the arithmetic hierarchy with the theory of Turing degrees. We omit the details as this will not be essentially used, and is a well understood idea.

Let $\mathcal{A}$ denote the first order language of arithmetic with non-logical symbols $\{0, +, \times, s, <\}$. (Or any chosen language of arithmetic sufficient to express Peano axioms.) Let $\mathcal{F}_0$ denote the set of Q -decidable formulas ϕ in arithmetic $\mathcal{A}$ with one free variable, where Q denotes the Robinson arithmetic. The preliminaries for the following are given in Section 2.1.

Definition 1.4 (cf. [5]). Given a first order theory F in any language $\mathcal{L}$, we say that F **1-consistent**, if:

- F interprets Q , that is $F \vdash^i Q$ as in Definition 2.5.
- For any formula $\phi \in \mathcal{F}_0$ the following holds:

$$(F_{\mathcal{A}} \vdash \exists m \phi(m)) \implies (\exists m F_{\mathcal{A}} \not\vdash \neg \phi(m)).$$

Here $F_{\mathcal{A}}$ is as in Notation 2.3. In other words, this says that $F_{\mathcal{A}}$ is 1-consistent.

We say that it is **2-consistent** if the same holds for Π_1 formulas ϕ with one free variable, more specifically formulas $\phi = \forall n g(m, n)$, with g Q -decidable.

The following theorems are naturally theorems of ZFC , with the proof predicate $\vdash$ being naturally interpreted. This theorem in particular tells us that the Gödel sentence is “computable” even for Σ_2 theories.

Theorem 1.5. There is a total computable map $\mathcal{G} : \mathcal{T} \rightarrow \mathcal{A}$, where the domain $\mathcal{T}$ is the set of Turing machines $\mathbb{N} \rightarrow \mathbb{N}$, and where $\mathcal{G}$ satisfies the following. For any $\mathcal{L}$ as above, suppose that T stably computes $M : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$. Let $F = M^s$ and also suppose that $F \vdash Q$. Then we have:

- (1) $F \not\vdash \mathcal{G}(T)$ if F is 1-consistent.
- (2) $F \not\vdash \neg \mathcal{G}(T)$ if F is 2-consistent.
- (3)

$$(1.6) \quad (F \text{ is 1-consistent}) \implies \mathcal{G}(T).$$

Furthermore, the Turing machine computing $\mathcal{G}$, can itself be given constructively.

As a corollary we get the following version of Gödel's second incompleteness theorem. In what follows, $\mathcal{Z}$ denotes the language of set theory. F is some $\mathcal{L}$ -theory such that $F \vdash^i ZFC$ and $F_{\mathcal{Z}}$ is as in Definition 2.3.

Corollary 1.7. *For F a theory in any language $\mathcal{L}$ such that $F \vdash^i ZFC$:*

$$\forall T \in \mathcal{T} (F \text{ is 1-consistent}) \wedge (F_{\mathcal{Z}} \vdash T \text{ stably enumerates } F) \implies (F_{\mathcal{Z}} \not\vdash F \text{ is 1-consistent}).$$

What follows is a stronger variant of the previous result, as we replace stable enumerability by a given Turing machine, by just the stable c.e. condition. However, we need to suppose that F is strongly consistent, (see Definition 5.11).

Theorem 1.8. *For F a theory in any language $\mathcal{L}$ such that $F \vdash^i ZFC$: if F is strongly consistent, (see Definition 5.11):*

$$(1.9) \quad (F_{\mathcal{Z}} \not\vdash F \text{ is 1-consistent}) \vee (F_{\mathcal{Z}} \not\vdash F \text{ is stably c.e., i.e. is } \Sigma_2).$$

1.2. Generalizations to Σ_n . There are natural candidates for how to generalize the theorem above. We may replace $M : \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$ by $M : \mathbb{N}^n \rightarrow \mathcal{A} \times \{\pm\}$, using this we can define a notion of n -stable computability, specializing to stable computability for $n = 1$. The above theorems should generalize to this setting of n -stable computability. In terms of arithmetic complexity this should be exactly the class Σ_{n+1} . We leave this for future developments.

2. SOME PRELIMINARIES

2.1. First order theories. Let $\mathcal{L}$ be a first order language. An $\mathcal{L}$ -theory F is a subset $F \subset \mathcal{L}$. F will be called deductively complete if it is closed under inference, that is if $F \vdash \alpha$ then $\alpha \in F$. Denote by $\overline{F}$ the deductive closure of F .

$$\overline{F} = \cap \{H \subset \mathcal{L} \mid H \text{ is deductively complete and } F \subset H\}.$$

We will need a notion of one first order theory interpreting another first order theory, possibly in a different language.

Definition 2.1. *Given first order languages $\mathcal{L}, \mathcal{L}'$, a **translation** is a set embedding $emb : \mathcal{L}' \rightarrow \mathcal{L}$ such that*

$$(\forall \alpha \in \mathcal{L}' \forall S \subset \mathcal{L}') S \vdash \alpha \implies emb(S) \vdash emb(\alpha).$$

*Crucially, we do not assume that emb is computable. We may call emb the **translation map**.*

Here is one well understood example.

Example 2.2. If $\mathcal{Z}$ denotes the first order language of set theory and $\mathcal{A}$ the first order language of arithmetic as previously, then there is a translation $emb_{\mathcal{A}, \mathcal{Z}} : \mathcal{A} \rightarrow \mathcal{Z}$. This map assigns to numerals in the language of arithmetic the Von Neumann naturals, e.g. we assign $\emptyset$ to 0. Moreover, $emb_{\mathcal{A}, \mathcal{Z}}$ is computable.

Definition 2.3. *Given a translation $emb : \mathcal{L}' \rightarrow \mathcal{L}$ and an $\mathcal{L}$ -theory F , we set $F_{\mathcal{L}'} = emb^{-1}(\overline{F}) \subset \mathcal{L}'$. To paraphrase, this is set of $\mathcal{L}'$ -sentences proved by F under the given translation.*

The following is immediate from the definitions.

Lemma 2.4. *For $\mathcal{L}, \mathcal{L}', emb, F$ as above $F_{\mathcal{L}'}$ is a deductively closed theory.*

Note that $F_{\mathcal{L}'}$ may not be $\mathcal{L}'$ -definable even if F is c.e., because emb is not assumed to be computable.

Definition 2.5. *Given an $\mathcal{L}$ -theory F and an $\mathcal{L}'$ -theory F' we write $F \vdash^i F'$ if there exists a translation $emb : \mathcal{L}' \rightarrow \mathcal{L}$ such that*

$$F_{\mathcal{L}'} \vdash F'$$

*In this case we say that F **interprets** F' .*

The following is also immediate from definitions.

Lemma 2.6. *Given first order languages $\mathcal{L}_0, \mathcal{L}_1, \mathcal{L}_2$ and an $\mathcal{L}_2$ -theory F_2 . If $F_2 \vdash^i F_1$ and $F_1 \vdash^i F_0$ then $F_2 \vdash^i F_0$.*

Corollary 2.7. $(F \vdash^i ZFC) \implies (F \vdash^i Q)$.

2.2. Abstractly encoded sets and abstract Turing machines. The material of this section will be used in the main argument. Working with encoded sets/maps as opposed to concrete subsets of $\mathbb{N}$ /functions will have some advantages as we can construct computable maps axiomatically. This approach is in essence the *standard* approach, but in the latter such encoding systems are usually implicit. We need to make it explicit, due to much complexity of the kind computable maps we need to construct. (We make explicit the axioms for the encodings, the latter themselves are still left implicit, as this is well understood.)

Definition 2.8. *We denote by $\mathcal{T}$ the set of all Turing machines $T : \mathbb{N} \rightarrow \mathbb{N}$. We write $*T(n)$ for the computation sequence of the Turing machine T with input n . As usual, for $T \in \mathcal{T}$, T also denotes the underlying partial function with $T(n) = m$ if $*T(n)$ halts with output m , and undefined otherwise.*

An **encoding** of a set A is at the moment just an injective set map $e : A \rightarrow \mathbb{N}$. But we will need to axiomatize this further. We extend the collection of encodings to a structure of a category.

In what follows, a map is a partial map, unless we specify that it is total. The category $\mathcal{S}$ will be a certain small “arrow category” whose objects are maps $e_A : A \rightarrow \mathbb{N}$, for e_A an embedding called **encoding map of A** , determined by a set A . More explicitly, the set of objects $\text{obj } \mathcal{S}$ of $\mathcal{S}$ consists of some set of pairs (A, e_A) where A is a set, and $e_A : A \rightarrow \mathbb{N}$ an embedding, determined by A . We may denote $e_A(A)$ by A_e .

We now describe the morphisms of $\mathcal{S}$.

Definition 2.9. *For $(N, e_N), (M, e_M)$ in $\text{obj } \mathcal{S}$. We say that $T \in \mathcal{T}$ **computes** a map $f : N \rightarrow M$ if T fits into a commutative diagram:*

$$\begin{array}{ccc} N & \xrightarrow{f} & M \\ \downarrow e_N & & \downarrow e_M \\ \mathbb{N} & \xrightarrow{T} & \mathbb{N}. \end{array}$$

*We say that $f : N \rightarrow M$ is **computable** if there exists a $T \in \mathcal{T}$ which computes f .*

Notation 1. We may just write $A \in \mathcal{S}$ for an object, with e_A implicit.

We call such an $A \in \mathcal{S}$ an **abstractly encoded set**. Then we set $\text{hom}_{\mathcal{S}}(N, M)$ to be the set of computable maps as above. Clearly, the composition of computable maps is computable so that $\mathcal{S}$ is a category we call a **Turing category**.

In addition, we ask that $\mathcal{S}$ satisfies the following axioms.

- (1) For $A \in \mathcal{S}$, the set A_e is computable (recursive). Here, as is standard, a set $S \subset \mathbb{N}$ is called *computable* if both S and its complement are computably enumerable, with S called *computably enumerable* if there is a computable partial function $\mathbb{N} \rightarrow \mathbb{N}$ with range S .
- (2) If $A, B \in \mathcal{S}$ then $A \times B \in \mathcal{S}$ and the projection maps $pr^A : A \times B \rightarrow A$, $pr^B : A \times B \rightarrow B$ are computable. Similarly for pr^B .
- (3) If $f : A \rightarrow B$ is computable, and $g : A \rightarrow C$ is computable then $A \rightarrow B \times C$, $a \mapsto (f(a), g(a))$ is computable. This combined with Axiom 2 implies that if $f : A \rightarrow B$, $g : C \rightarrow D$ is computable then the map $A \times B \rightarrow C \times D$, $(a, b) \mapsto (f(a), g(b))$ is computable.
- (4) The set $\mathbb{N}$ has the identity encoding $e_{\mathbb{N}} : \mathbb{N} \rightarrow \mathbb{N}$.
- (5) The set $\mathcal{T}$ is encoded. The partial map

$$U : \mathcal{T} \times \mathbb{N} \rightarrow \mathbb{N},$$

$U(T, \Sigma) := T(\Sigma)$ whenever $*T(\Sigma)$ halts and undefined otherwise, is computable. We can understand a Turing machine computing U as the “universal Turing machine”.

- (6) The next axiom gives a prescription for construction of Turing machines. Let $A, B, C \in \mathcal{S}$, and suppose that $f : A \times B \rightarrow C$ is computable. Let $f^a : B \rightarrow C$ be the map $f^a(b) = f(a, b)$. Then there is a computable map

$$s : A \rightarrow \mathcal{T}$$

so that for each a $s(a)$ computes f^a .

- (7) If $(A, e_A) \in \mathcal{S}$ and $B \subset A$ s.t. $e_A(B)$ is computable, then B is encoded. We call this the *induced encoding*.
- (8) The final axiom is for utility. If $A \in \mathcal{S}$ then $L(A) \in \mathcal{S}$, where

$$L(A) = \bigcup_{n \in \mathbb{N}} \text{Maps}(\{0, \dots, n\}, A),$$

and $\text{Maps}(\{0, \dots, n\}, A)$ denotes the set of total maps. We also have:

- (a) Let $A \in \mathcal{S}$ and let

$$\text{length} : L(A) \rightarrow \mathbb{N},$$

be the length function, s.t. for $l \in L(A)$, $l : \{0, \dots, n\} \rightarrow A$, $\text{length}(l) = n$. Then length is computable.

- (b) Define

$$P : L(A) \times \mathbb{N} \rightarrow A,$$

$$P(l, i) := \begin{cases} l(i), & \text{if } 0 \leq i \leq \text{length}(l) \\ \text{undefined}, & \text{for } i > \text{length}(l). \end{cases}$$

Then P is computable.

- (c) For $A, B \in \mathcal{S}$ and $f : A \rightarrow L(B)$ a partial map, suppose that:

- The partial map $A \times \mathbb{N} \rightarrow B$, $(a, n) \mapsto P(f(a), n)$ is computable.
- The partial map $A \rightarrow \mathbb{N}$, $a \mapsto \text{length}(f(a))$ is computable.

Then f is computable.

Lemma 2.10. *If $f : A \rightarrow B$ is computable then the map $L(f) : L(A) \rightarrow L(B)$,*

$$l \mapsto \begin{cases} i \mapsto f(l(i)), & \text{if } f(l(i)) \text{ is defined for all } 0 \leq i \leq \text{length}(l) \\ \text{undefined}, & \text{otherwise,} \end{cases}$$

is computable. Also, the map $LU : \mathcal{T} \times L(\mathcal{U}) \rightarrow L(\mathcal{U})$,

$$l \mapsto \begin{cases} i \mapsto U(T, (l(i))), & \text{if } U(T, (l(i))) \text{ is defined for all } 0 \leq i \leq \text{length}(l) \\ \text{undefined}, & \text{otherwise} \end{cases}$$

is computable.

Proof. This is just a straightforward application of the axioms and Axiom 8 in particular. We leave the details as an exercise. $\square$

The above axioms suffice for our purposes, but there are a number of possible extensions (dealing with other set theoretic constructions like the set theoretic sum). The specific such category $\mathcal{S}$ that we need will be clear from context later on. We only need to encode finitely many basic types sets. (The rest are determined from the product, and list constructors, etc.) For example, aside from $\mathbb{N}, \mathcal{T}$, the category $\mathcal{S}$ should contain $\mathcal{A}, \mathcal{L}, \{\pm\}$, with $\{\pm\}$ a set with two elements $+, -$. The main naturality properties for the encoding of $\mathcal{T}$ are already stated as Axioms 5, 6. We can of course take the standard type of encoding for $\mathcal{T}$. We also fix the standard Gödel encoding of $\mathcal{A}$. The only naturality property for $\mathcal{A}$ that is that certain natural maps of the type (4.8) are computable. This is satisfied for the standard encoding.

The fact that such Turing categories $\mathcal{S}$ exist is a folklore theorem of computer science starting with the foundational work of Gödel, Turing and others. For example, Axiom 6 corresponds to the “s-m-n theorem” Soare [14, Theorem 1.5.5], which works as follows. Given a classical 2-input Turing machine

$$T : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N},$$

there is a Turing machine $s_T : \mathbb{N} \rightarrow \mathbb{N}$ s.t. for each m $s_T(m)$ is the Turing-Gödel encoding natural, of a Turing machine computing the map $f^m : n \mapsto T(m, n)$.

In modern terms, the construction of $\mathcal{S}$ is essentially a part of a definition of a computer programming language, with algebraic data types, e.g. Haskell.

3. STABLE COMPUTABILITY AND ARITHMETIC

In this section, general sets, often denoted as B , are intended to be encoded. And all maps are partial maps, unless specified otherwise.

Definition 3.1. *Given a map:*

$$M : \mathbb{N} \rightarrow B \times \{\pm\},$$

We say that $b \in B$ is **M -stable** if there is an m with $M(m) = (b, +)$ and there is no $n > m$ with $M(n) = (b, -)$.

Definition 3.2. *Given a map*

$$M : \mathbb{N} \rightarrow B \times \{\pm\},$$

we define

$$M^s \subset B$$

*to be the set of all the M -stable b . We call this the **stabilization of M** . When M is morphism in $\mathcal{S}$, we say that $S \subset B$ is **stably c.e.** if $S = M^s$. We say that $T \in \mathcal{T}$ **stably computes** $M : \mathbb{N} \rightarrow B \times \{\pm\}$, if it computes $N : \mathbb{N} \rightarrow B \times \{\pm\}$, s.t. $M^s = N^s$.*

In general M^s may not be computable even if M is computable. Explicit examples of this sort can be readily constructed as shown in the following.

Example 3.3. Let Pol denote the set of all Diophantine polynomials, abstractly encoded. We can construct a total computable map

$$A : \mathbb{N} \rightarrow Pol \times \{\pm\}$$

whose stabilization consists of all Diophantine (integer coefficients) polynomials with no integer roots. Similarly, we can construct a computable map D whose stabilization consists of pairs (T, n) for $T : \mathbb{N} \rightarrow \mathbb{N}$ a Turing machine and $n \in \mathbb{N}$ such that $*T(n)$ does not halt.

In the case of Diophantine polynomials, here is an (inefficient) example. Fixing a suitable encoding of $\mathbb{Z}$. Let

$$Z : \mathbb{N} \rightarrow Pol, \quad N : \mathbb{N} \rightarrow \mathbb{Z}$$

be total bijective computable maps. The encoding of $Pol, \mathbb{Z}$ should be suitably natural so that in particular the map

$$E : \mathbb{Z} \times Pol \rightarrow \mathbb{Z}, \quad (n, p) \mapsto p(n)$$

is computable. In what follows, for each $n \in \mathbb{N}$, $A_n \in L(Pol \times \{\pm\})$. $\cup$ will be here and elsewhere in the paper the natural list union operation. More specifically, if

$$l_1 : \{0, \dots, n\} \rightarrow B, \quad l_2 : \{0, \dots, m\} \rightarrow B$$

are two lists then $l_1 \cup l_2$ is defined by:

$$(3.4) \quad l_1 \cup l_2(i) = \begin{cases} l_1(i), & \text{if } i \in \{0, \dots, n\} \\ l_2(i - n - 1), & \text{if } i \in \{n + 1, \dots, n + m + 1\} \end{cases}.$$

If $B \in \mathcal{S}$, it is easy to see by the axioms of $\mathcal{S}$ that

$$\cup : L(B) \times L(B) \rightarrow L(B), \quad (l, l') \mapsto l \cup l'$$

is computable.

For $n \in \mathbb{N}$ define A_n recursively by: $A_0 := \emptyset$,

$$A_{n+1} := A_n \cup \bigcup_{m=0}^n (Z(m), d^n(Z(m))),$$

where $d^n(p) = +$ if none of $\{N(0), \dots, N(n)\}$ are roots of p , $d^n(p) = -$ otherwise.

Note that

$$(\forall n \in \mathbb{N}) A_{n+1}|_{\text{domain } A_n} = A_n, \text{ and } \text{length}(A_{n+1}) > \text{length}(A_n),$$

so we may define $A(n) := A_{n+1}(n)$. With this definition $A(\mathbb{N}) = \cup_{n \in \mathbb{N}} \text{image}(A_n)$.

Since E is computable, utilizing the axioms, and the recursive program above, it can be explicitly verified that A is computable. Moreover, by construction the stabilization A^s consists of all Diophantine polynomials that have no integer roots.

3.1. Decision maps. By a *decision map*, we mean a map of the form:

$$D : B \times \mathbb{N} \rightarrow \{\pm\}.$$

This kind of maps will play a role in our arithmetic incompleteness theorems, and we now develop some of their theory.

Definition 3.5. Let $B \in \mathcal{S}$, define $\mathcal{D}_B$ to be the set of $T \in \mathcal{T}$ s.t. exists $T' : B \times \mathbb{N} \rightarrow \{\pm\}$, and a commutative diagram:

$$\begin{array}{ccc} B \times \mathbb{N} & \xrightarrow{T'} & \{\pm\} \\ \downarrow e_{B \times \mathbb{N}} & & \downarrow e_{\{\pm\}} \\ \mathbb{N} & \xrightarrow{T} & \mathbb{N}. \end{array}$$

More concretely, this is the set of T s.t.:

$$(\forall n \in \text{image } e_{B \times \mathbb{N}} \subset \mathbb{N}) (T(n) \in \text{image } e_{B \times \{\pm\}} \text{ or } T(n) \text{ is undefined.})$$

As $e_{\{\pm\}}$ is injective, T' above is uniquely determined *if* it exists. From now on, for $T \in \mathcal{D}_B$, when we write T' it is meant to be of the form above.

First we will explain one construction of elements of $\mathcal{D}_B$. From Turing machines of the following form.

Definition 3.6. Let $B \in \mathcal{S}$. Define $\mathcal{T}_B$ to be the set of $T \in \mathcal{T}$ s.t. exists $T' : \mathbb{N} \rightarrow B \times \{\pm\}$, and a commutative diagram:

$$\begin{array}{ccc} \mathbb{N} & \xrightarrow{T'} & B \times \{\pm\} \\ \downarrow e_{\mathbb{N}} & & \downarrow e_{B \times \{\pm\}} \\ \mathbb{N} & \xrightarrow{T} & \mathbb{N}. \end{array}$$

From now on, given $T \in \mathcal{T}_B$, if we write T' then it is will be assumed to be of the form above. As before, it is uniquely determined when exists.

Lemma 3.7. For $(A, e_A) \in \mathcal{S}$ define the map:

$$(3.8) \quad e_A^{-1} : \mathbb{N} \rightarrow A,$$

defined by

$$e_A^{-1}(n) = \begin{cases} e_A^{-1}(n), & \text{if } n \in \text{image } e_A \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

Then e_A^{-1} is computable.

Proof. Let $T \in \mathcal{T}$ compute the map defined by

$$n \mapsto \begin{cases} n, & \text{if } n \in \text{image } e_A \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

As $\text{image } e_A$ is decidable T does exist.

Then clearly we have a commutative diagram:

$$\begin{array}{ccc} \mathbb{N} & \xrightarrow{e_A^{-1}} & A \\ \downarrow id & & \downarrow e_A \\ \mathbb{N} & \xrightarrow{T} & \mathbb{N}. \end{array}$$

□

Lemma 3.9. Let $B \in \mathcal{S}$. There is a computable total map

$$K_B : \mathcal{T} \rightarrow \mathcal{T},$$

with the properties:

- (1) For each T , $K_B(T) \in \mathcal{T}_B$ and is total if T is total.
- (2) If $T \in \mathcal{T}_B$ then $K_B(T)$ and T compute the same maps $\mathbb{N} \rightarrow B \times \{\pm\}$.

Proof. Let $G : \mathcal{T} \times \mathbb{N} \rightarrow B \times \{\pm\}$ be the composition of the sequence of maps

$$\mathcal{T} \times \mathbb{N} \xrightarrow{U} \mathbb{N} \xrightarrow{e_{B \times \{\pm\}}^{-1}} B \times \{\pm\}.$$

Hence, G is a composition of computable maps and so is computable. By Axiom 6 there is an induced computable map $K_B : \mathcal{T} \rightarrow \mathcal{T}$ so that $K_B(T)$ is the encoding of $G^T : \mathbb{N} \rightarrow B \times \{\pm\}$, $G^T(n) = G(T, n)$. By construction, if $T \in \mathcal{T}_B$ then $T' = (K_B(T))'$. So that we are done. $\square$

3.1.1. Constructing decision Turing machines. We will need a few preliminaries as we need to deal with the following issue. For a given Turing machine $\mathbb{N} \rightarrow B \times \{\pm\}$, we may construct a total Turing machine with the same image, but not preserving stability conditions. So we must adjust the totalization procedure to keep track of the stability conditions.

The following is well known:

Lemma 3.10. *There is a computable map $Tot : \mathcal{T} \rightarrow \mathcal{T}$ s.t. $\forall T \in \mathcal{T}$:*

- (1) $Tot(T)$ is total for each T .
- (2) $\text{image}(Tot(T)) = \text{image } T$.

Let

$$\begin{aligned} \pi_{\mathbb{N}} : \mathbb{N} \times B \times \{\pm\} &\rightarrow \mathbb{N} \\ \pi_{B \times \mathbb{N}} : \mathbb{N} \times B \times \{\pm\} &\rightarrow B \times \{\pm\} \end{aligned}$$

be the natural projections. We say that a subset $S \subset \mathbb{N} \times B \times \{\pm\}$ is *graphical*, if $\pi_{\mathbb{N}}^{-1}(n)$ is at most one element for each $n \in \mathbb{N}$.

On a graphical $S \subset \mathbb{N} \times B \times \{\pm\}$ there is a natural total order $s \leq s'$ if $\pi_{\mathbb{N}}(s) \leq \pi_{\mathbb{N}}(s')$. So a finite graphical $S \subset \mathbb{N} \times B \times \{\pm\}$ determines a list $l_S \in L(B \times \{\pm\})$ by

$$(3.11) \quad l_S(i) = \pi_{B \times \{\pm\}}(s_i),$$

where s_i is the $i - 1$ element of S with respect to the total order.

Definition 3.12. *Let $l \in L(B \times \{\pm\})$. Define $b \in B$ to be **l -stable** if there is an $m \leq \text{length}(l)$ s.t. $l(m) = (b, +)$ and there is no $m < k \leq \text{length}(l)$ s.t. $l(k) = (b, -)$.*

Given a map $f : \mathbb{N} \rightarrow B \times \{\pm\}$ let $\text{gr } f : \mathbb{N} \rightarrow \mathbb{N} \times B \times \{\pm\}$ be the map $\text{gr } f(n) = (n, f(n))$. Expanding on Lemma 3.9, there is a total computable map:

$$(3.13) \quad Gr_B : \mathcal{T} \rightarrow \mathcal{T},$$

with the property: $Gr_B(T)$ computes the map $\text{gr}(K_B(T)')$.

Set

$$(3.14) \quad f^T := e_{\mathbb{N} \times B \times \{\pm\}}^{-1} \circ Tot \circ Gr_B(T) \circ e_{\mathbb{N}}.$$

So that $f^T : \mathbb{N} \rightarrow \mathbb{N} \times B \times \{\pm\}$ and is total.

Finally, define

$$G : B \times \mathcal{T} \times \mathbb{N} \rightarrow \{\pm\}$$

to be the map:

$$G(b, T, n) = \begin{cases} +, & b \text{ is } l_S\text{-stable, for } S = \text{image}(f^T|_{\{0, \dots, n\}}). \\ -, & \text{otherwise.} \end{cases}$$

Lemma 3.15. *G is computable.*

Proof. This is clear, but we make this explicit. Let

$$(3.16) \quad g : \mathbb{N} \rightarrow L(\mathbb{N})$$

be the map $g(n) = \{0, \dots, n\}$, it is clearly computable directly by the Axiom 8. Let

$$L^{graph}(\mathbb{N} \times B \times \{\pm\}) \subset L(\mathbb{N} \times B \times \{\pm\})$$

consist of lists l s.t. $S = \text{image } l$ is graphical. The latter subset has the induced encoding, see Axiom 7.

Then we can express G as the composition of the sequence of maps:

$$\begin{aligned} B \times \mathcal{T} \times \mathbb{N} &\xrightarrow{id \times (Gr_B) \times g} B \times \mathcal{T} \times L(\mathbb{N}) \xrightarrow{id \times L(U)} B \times L(\mathbb{N}) \\ &\xrightarrow{id \times L(e_{\mathbb{N} \times B \times \{\pm\}}^{-1})} B \times L^{graph}(\mathbb{N} \times B \times \{\pm\}) \xrightarrow{id \times ord} B \times L(B \times \{\pm\}) \rightarrow \{\pm\}, \end{aligned}$$

where

- (1) $ord : L^{graph}(\mathbb{N} \times B \times \{\pm\}) \rightarrow L(B \times \{\pm\})$ is the map $ord(l) = l_S$, for $S = \text{image } l$.
- (2) The last map is:

$$(b, l) \mapsto \begin{cases} +, & \text{if } b \text{ is } l\text{-stable} \\ -, & \text{otherwise.} \end{cases}$$

The latter two maps are computable by explicit verification, utilizing the axioms. In particular all the maps in the composition are computable and so G is computable. □

Let

$$(3.17) \quad Dec_B : \mathcal{T} \rightarrow \mathcal{T},$$

be the computable total map corresponding G via Axiom 6, so that $Dec_B(T)$ is the Turing machine computing

$$G^T : B \times \mathbb{N} \rightarrow \{\pm\}, \quad G^T(b, n) = G(b, T, n).$$

The following is immediate from the construction:

Lemma 3.18. *$Dec_B(T)$ has the property:*

$$\forall T \in \mathcal{T} \quad Dec_B(T) \in \mathcal{D}_B.$$

Furthermore, $Dec_B(T)$ is total.

Definition 3.19. *For a map $D : B \times N \rightarrow \{\pm\}$, we say that $b \in B$ is **D-decided** if there is an m s.t. $D(b, m) = +$ and for all $n \geq m$ $D(b, n) \neq -$. Likewise, for $T \in \mathcal{D}_B$ we say that $b \in B$ is **T-decided** if it is T' -decided. Also for $T \in \mathcal{T}_B$ we say that b is **T-stable** if it is T' -stable in the sense of Definition 3.1.*

Lemma 3.20. *Suppose that $T \in \mathcal{T}_B$ then b is T -stable iff b is $Dec_B(T)$ -decided.*

Proof. Suppose that b is T -stable. In particular, there is an $m \in \mathbb{N}$ so that $T(m) = (b, +)$ and there is no $n > m$ so that $T(m) = (b, -)$. By construction, there exists an m' s.t. $f^T(m') = (m, b, +)$. It is then immediate that b is l_S -stable, for any $S_n = \text{image}(f^T|_{\{0, \dots, n\}})$ s.t. $n \geq m'$.

Then by construction of G , $G(b, T, m') = +$ and there is no $n > m'$ s.t. $G(b, T, m') = -$. Thus b is G^T -decided and so $Dec_B(T)$ -decided.

The converse is also clear. Suppose b is G^T decided. Let N be s.t.

(1) $G(b, T, N) = +$.

(2) There is no $n > N$ s.t. $G(b, T, n) = -$.

By [1](#) there is an $n_0 \in \{0, \dots, N\}$ s.t. $f^T(n_0) = (n_b, b, +)$ and there is no $n \in \{0, \dots, N\}$ with $pr_{\mathbb{N}} \circ f^T(n) > n_b$ s.t. $pr_{B \times \{\pm\}} \circ f^T(n) = (b, -)$.

Now, if b is not T -stable then there is a $n' > n_b$ s.t. $T(n') = (b, -)$. By the previous paragraph,

$$n' \notin \text{image } pr_{\mathbb{N}} \circ f^T|_{\{0, \dots, N\}}.$$

So there is an $n > N$ so that $f^T(n) = (n', b, -)$. But then $G(b, T, n) = -$ which contradicts [2](#). $\square$

Example 3.21. By the [Example 3.3](#) above there is a computable map

$$P = Dec_{Pol}(A) : Pol \times \mathbb{N} \rightarrow \{\pm\}$$

that stably soundly decides if a Diophantine polynomial has integer roots, meaning:

$$p \text{ is } P\text{-decided} \iff p \text{ has no integer roots.}$$

We may similarly, stably solve the halting problem, in this sense.

Definition 3.22. *Given a pair of maps*

$$M_0 : B \times \mathbb{N} \rightarrow \{\pm\}$$

$$M_1 : B \times \mathbb{N} \rightarrow \{\pm\},$$

we say that they are stably equivalent if

$$b \text{ is } M\text{-decided} \iff b \text{ is } M'\text{-decided.}$$

If $T \in \mathcal{D}_B$ then we say that T stably computes M iff T' is stably equivalent to M .

3.2. Decision maps in formal systems. Let $\mathcal{L}$ be as in the Introduction. Let $\mathcal{T}_{\mathcal{L}}$ be as in [Definition 3.6](#) with respect to $B = \mathcal{L}$. The following is a version for stably c.e. theories of the classical fact, going back to Gödel, that for a theory with a c.e. set of axioms we may computably enumerate its theorems. Moreover, the procedure to obtain the corresponding Turing machine is computably constructive.

Notation 2. Note that each $T \in \mathcal{T}_{\mathcal{L}}$, determines the set

$$(T')^s \subset \mathcal{L},$$

called the stabilization of T' , we hereby abbreviate the notation for this set as T^s .

Lemma 3.23. *There is a computable total map:*

$$C : \mathcal{T} \rightarrow \mathcal{T}$$

so that $\forall T \in \mathcal{T} : C(T) \in \mathcal{T}_{\mathcal{L}}$. If in addition $T \in \mathcal{T}_{\mathcal{L}}$ then $(C(T))^s$ is the deductive closure of T^s .

Proof. Let $L(\mathcal{L})$ be the list construction on $\mathcal{L}$ as previously. The following lemma is classical and its proof is omitted.

Lemma 3.24. *There is a total computable map:*

$$\Phi : L(\mathcal{L}) \times \mathbb{N} \rightarrow \mathcal{L}$$

with the following property. For each $l \in L(\mathcal{L})$, $\Phi(\{l\} \times \mathbb{N})$ is the deductive closure of the theory $F_l = \text{image } l$.

Using the map ord from Lemma 3.15, define a map

$$\zeta : L(\mathcal{L}) \times L^{graph}(\mathbb{N} \times \mathcal{L} \times \{\pm\}) \rightarrow \{\pm\}$$

by

$$\zeta(l, l') = \begin{cases} +, & \text{if for each } 0 \leq i \leq \text{length}(l), l(i) \text{ is } ord(l')\text{-stable.} \\ -, & \text{otherwise.} \end{cases}$$

Utilizing the axioms of $\mathcal{S}$ we readily see that ζ is computable.

Now define H to be the composition of the sequence of maps:

$$\mathcal{T} \times L(\mathbb{N}) \xrightarrow{Gr_{\mathcal{L}} \times id} \mathcal{T} \times L(\mathbb{N}) \xrightarrow{LU} L(\mathbb{N}) \xrightarrow{L(e_{\mathbb{N} \times \mathcal{L} \times \{\pm\}}^{-1})} L^{graph}(\mathbb{N} \times \mathcal{L} \times \{\pm\}).$$

All the maps in the composition are computable directly by the axioms of $\mathcal{S}$ and so H is computable.

We may now construct our map C . In what follows $\cup$ will be the natural list union operation as previously in (3.4). Set

$$L_n(\mathbb{N}) := \{l \in L(\mathbb{N}) \mid \max l \leq n, \max l \text{ the maximum of } l \text{ as a map}\}.$$

Let

$$pr_{\mathcal{L}} : \mathbb{N} \times \mathcal{L} \times \{\pm\} \rightarrow \mathcal{L}$$

be the natural projection. For $n \in \mathbb{N}$, define $U_n^T \in L(\mathcal{L} \times \{\pm\})$ recursively by $U_0^T := \emptyset$,

$$U_{n+1}^T := U_n^T \cup \bigcup_{l \in L_{n+1}(\mathbb{N})} \bigcup_{0 \leq m \leq n+1} (\Phi(L_{pr_{\mathcal{L}}} \circ H(T, l), m), \zeta(L_{pr_{\mathcal{L}}} \circ H(T, l), H(T, \{0, \dots, n+1\}))).$$

As in Example 3.3 we define

$$U^T : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}, \quad U^T(n) := U_{n+1}^T(n).$$

And this induces a total map

$$U : \mathcal{T} \times \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\},$$

$U(T, n) := U^T(n)$. U is computable by explicit verification, utilizing the axioms of $\mathcal{S}$, and the recursive program for $\{U_n^T\}$. Hence, by the Axiom 6 there is an induced by U computable total map:

$$C : \mathcal{T} \rightarrow \mathcal{T},$$

s.t. for each $T \in \mathcal{T}$ $C(T)$ computes U^T .

By construction, this has the needed properties, and we are done. $\square$

Notation 3. Let $M : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$ be a map. To simplify notation we use from now the shorthand: $M \vdash \alpha$, for $M_{\mathcal{A}}^s \vdash \alpha$, whenever $\alpha \in \mathcal{A}$, and whenever this may cause no confusion.

Definition 3.25. Let $\mathcal{F}_0$, as in the introduction, denote the set of formulas ϕ of arithmetic with one free variable so that $\phi(n)$ is an Q -decidable sentence for each n . Let $M : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$ be a map. We say that M is **speculative** if the following holds. Let $\phi \in \mathcal{F}_0$, and set

$$(3.26) \quad \alpha_{\phi} = \forall m \phi(m),$$

then

$$(\forall m Q \vdash \phi(m)) \implies M \vdash \alpha_{\phi}.$$

Note that of course the left-hand side is not the same as $Q \vdash \alpha_{\phi}$.

Remark 3.27. In a more broad context this condition is sometimes called the ω -rule for a theory.

Note that we previously constructed an Example 3.3 of a Turing machine, with an analogue of this speculative property. Moreover, we have the following crucial result, which to paraphrase states that there is an operation $Spec$ that converts a stably c.e. theory to a speculative stably c.e. theory, at a certain loss of consistency.

Theorem 3.28. *There is a computable total map $Spec : \mathcal{T} \rightarrow \mathcal{T}$, with the following properties:*

- (1) $\text{image } Spec \subset \mathcal{T}_A$.
- (2) Let $T \in \mathcal{T}_A$. Set $T_{spec} = Spec(T)$ then T'_{spec} is speculative, moreover if T' is total then so is T'_{spec} .
- (3) Using Notation 2, if $T \in \mathcal{T}_A$ then $T_{spec}^s \supset T^s$.
- (4) If $T \in \mathcal{T}_A$ and T^s is 1-consistent then T_{spec}^s is consistent.

Proof. $\mathcal{F}_0, \mathcal{A}$ are assumed to be encoded so that the map

$$ev : \mathcal{F}_0 \times \mathbb{N} \rightarrow \mathcal{A}, \quad (\phi, m) \mapsto \phi(m)$$

is computable. We then need:

Lemma 3.29. *There is a total computable map $F : \mathbb{N} \rightarrow \mathcal{F}_0 \times \{\pm\}$ with the property:*

$$F^s = G := \{\phi \in \mathcal{F}_0 \mid \forall m \ Q \vdash \phi(m)\}.$$

Proof. The construction is analogous to the construction in the Example 3.3 above. Fix any total, bijective, Turing machine

$$Z : \mathbb{N} \rightarrow \mathcal{F}_0.$$

For a $\phi \in \mathcal{F}_0$ we will say that it is *n-decided* if

$$(\forall m \in \{0, \dots, n\}) \ Q \vdash \phi(m).$$

In what follows each F_n has the type of ordered finite list of elements of $\mathcal{F}_0 \times \{\pm\}$, and $\cup$ will be the natural list union operation, as previously. Define $\{F_n\}_{n \in \mathbb{N}}$ recursively by $F_0 := \emptyset$,

$$F_{n+1} := F_n \cup \bigcup_{\phi \in \{Z(0), \dots, Z(n)\}} (\phi, d^n(\phi)),$$

where $d^n(\phi) = +$ if ϕ is n -decided and $d^n(\phi) = -$ otherwise.

We set $F(n) := F_{n+1}(n)$. This is a total map

$$F : \mathbb{N} \rightarrow \mathcal{F}_0 \times \{\pm\},$$

having the property $F(\mathbb{N}) = \cup_n \text{image}(F_n)$. F is computable by explicit verification, using the axioms of $\mathcal{S}$. $\square$

Returning to the proof of the theorem. Let $K = K_A : \mathcal{T} \rightarrow \mathcal{T}$ be as in Lemma 3.9. For $\phi \in \mathcal{F}_0$ let α_ϕ be as in (3.26). Define: $H : \mathcal{T} \times \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$ by

$$H(T, n) := \begin{cases} (K_A(T))'(k), & \text{if } n = 2k + 1 \\ (\alpha_{pr_{\mathcal{F}_0} \circ F(k)}, pr_{\pm} \circ F(k)), & \text{if } n = 2k, \end{cases}$$

where $pr_{\mathcal{F}_0} : \mathcal{F}_0 \times \{\pm\} \rightarrow \mathcal{F}$, and $pr_{\pm} : \mathcal{F}_0 \times \{\pm\} \rightarrow \{\pm\}$ are the natural projections. H is computable directly by the axioms of $\mathcal{S}$. (Factor H as a composition of computable maps as previously.)

Let $Spec : \mathcal{T} \rightarrow \mathcal{T}$ be the computable map corresponding to H via Axiom 6. In particular, for each $T \in \mathcal{T}$, $Spec(T)$ computes the map

$$T'_{spec} := H^T : \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}, \quad H^T(n) = H(T, n),$$

which by construction is speculative. Now, $Spec(T)$ satisfies the Properties 1, 2, 3 immediately by construction. It only remains to check Property 4.

In what follows, let $T \in \mathcal{T}_A$, and set $F_{spec} = (T'_{spec})^s$, and $F = (T')^s$.

Lemma 3.30. F_{spec} consistent unless for some $\phi \in G$

$$F \vdash \neg \forall m \phi(m).$$

Proof. Suppose that F_{spec} is inconsistent so that:

$$F \cup \{\alpha_{\phi_1}, \dots, \alpha_{\phi_n}\} \vdash \alpha \wedge \neg \alpha$$

for some $\alpha \in \mathcal{A}$, and some $\phi_1, \dots, \phi_n \in G$. Hence,

$$F \vdash \neg(\alpha_{\phi_1} \wedge \dots \wedge \alpha_{\phi_n}).$$

But

$$\alpha_{\phi_1} \wedge \dots \wedge \alpha_{\phi_n} \iff \forall m \phi(m),$$

where ϕ is the formula with one free variable: $\phi(m) := \phi_1(m) \wedge \dots \wedge \phi_n(m)$. Clearly $\phi \in G$, since $\phi_i \in G$, $i = 1, \dots, n$. Hence, the conclusion follows. $\square$

Suppose that F_{spec} inconsistent, then by the lemma above for some $\phi \in G$:

$$F \vdash \exists m \neg \phi(m).$$

By the hypothesis of Property 4, F is 1-consistent and so:

$$\exists m F \not\vdash \phi(m).$$

But ϕ is in G , and $F \vdash Q$ so that $\forall m F \vdash \phi(m)$ and so

$$\exists m F \vdash \neg \phi(m) \wedge \phi(m).$$

So F is inconsistent, a contradiction, so F_{spec} is consistent. $\square$

4. THE STABLE HALTING PROBLEM

Let $\mathcal{D}_T \subset \mathcal{T}$ be as in Definition 3.5 with respect to $B = \mathcal{T}$.

Definition 4.1. For $T \in \mathcal{D}_T$, T is **T -decided**, is a special case of Definition 3.19. Or more specifically, it means that the element $T \in \mathcal{T}$ is T' -decided. We also say that T is not T -decided, when $\neg(T \text{ is } T\text{-decided})$ holds.

We call a map $D : \mathcal{T} \times \mathbb{N} \rightarrow \{\pm\}$ a **Turing decision map**. In what follows, denote by $s(T)$ the sentence:

$$(T \in \mathcal{D}_T) \wedge (T \text{ is not } T\text{-decided}).^3$$

Definition 4.2. We say a Turing decision map D is **stably sound on $T \in \mathcal{T}$** if

$$(T \text{ is } D\text{-decided}) \implies s(T).$$

We say that D is **stably sound** if it is stably sound on all T . We say that D **stably decides T** if:

$$s(T) \implies T \text{ is } D\text{-decided}.$$

We say that D **stably soundly decides T** if D is stably sound on T and D stably decides T . We say that D is **stably sound and complete** if D stably soundly decides T for all $T \in \mathcal{T}$.

³Written more formally this is the sentence $(T \in \mathcal{D}_T) \wedge ((T \in \mathcal{D}_T) \implies (T \text{ is not } T\text{-decided}))$, we will sometimes use this kind of contraction, and this will no longer be mentioned.

The informal interpretation of the above is that each such D is understood as an operation with the properties:

- For each T, n $D(T, n) = +$ if and only if D “decides” the sentence $s(T)$ is true, at the moment n .
- For each T, n $D(T, n) = -$ if and only if D cannot “decide” the sentence $s(T)$ at the moment n , or D “decides” that $s(T)$ is false.

In what follows for $T \in \mathcal{T}$, and D as above, $\Theta_{D,T}$ is shorthand for the sentence:

$$T \text{ stably computes } D.$$

Lemma 4.3. *If D is stably sound on $T \in \mathcal{T}$ then*

$$\neg\Theta_{D,T} \vee \neg(T \text{ is } D\text{-decided}).$$

Proof. If T is D -decided then since D is stably sound on T , $T \in \mathcal{D}_{\mathcal{T}}$ and T is not T -decided. So if in addition $\Theta_{D,T}$ then T is not D -decided a contradiction. $\square$

The following is the “stable” analogue of Turing’s halting theorem.

Theorem 4.4. *There is no (stably) computable Turing decision map D that is stably sound and complete.*

Proof. Let D be stably sound and complete. Then by the above lemma we obtain:

$$(4.5) \quad \forall T \in \mathcal{T} : \Theta_{D,T} \vdash \neg(T \text{ is } D\text{-decided}).$$

But it is immediate:

$$(4.6) \quad \forall T \in \mathcal{T} (\Theta_{D,T} \implies (\neg(T \text{ is } D\text{-decided})) \implies \neg(T \text{ is } T\text{-decided})).$$

So combining (4.5), (4.6) above we obtain

$$\forall T \in \mathcal{T} (\Theta_{D,T} \implies \neg(T \text{ is } T\text{-decided})).$$

But D is complete so $(T \in \mathcal{D}_{\mathcal{T}}) \wedge \neg(T \text{ is } T\text{-decided}) \implies T \text{ is } D\text{-decided}$ and so:

$$\forall T \in \mathcal{T} (\Theta_{D,T} \implies (T \text{ is } D\text{-decided})).$$

Combining with (4.5) we get

$$\forall T \in \mathcal{T} \neg\Theta_{D,T},$$

which is what we wanted to prove. $\square$

Theorem 4.7. *Suppose $F \subset \mathcal{A}$ is stably c.e. and sound theory, then there is a constructible (given a Turing machine stably computing F) true in the standard model of arithmetic sentence $\alpha(F)$, which F does not prove.*

The fact that such an $\alpha(F)$ exists, can be immediately deduced from Tarski undecidability of truth, as the set F must be definable in first order arithmetic by the condition that F is stably c.e. However, our sentence is constructible and elementary. Moreover, the basic form of this sentence will be used in the next section. The above theorem is partly based on meta logic. This is in sharp contrast to the syntactic incompleteness theorems in the following section which are actual theorems of *ZFC*.

Proof of Theorem 4.7. Suppose that F is stably c.e. and is sound. Let $M : \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$ be a total computable map s.t. $F = M^s(N)$. Let $C(M_e)$ be as in Lemma 3.23, where M_e computes M . If we understand arithmetic as being embedded in set theory ZFC in the standard way, then for each $T \in \mathcal{T}$ the sentence $s(T)$ is logically equivalent in ZFC to a first order sentence in arithmetic, that we also call $s(T)$, slightly abusing notation. The corresponding translation map

$$(4.8) \quad s : \mathcal{T} \rightarrow \mathcal{A}, T \mapsto s(T)$$

is computable. Indeed, this kind of translation already appears in the original work of Turing [1].

Define a Turing decision map D by

$$D(T, n) := (Dec_{\mathcal{A}}(C(M_e)))'(s(T), n)$$

for $Dec_{\mathcal{A}}$ as in (3.17) defined with respect to $B = \mathcal{A}$, and where C is as in Section 3. Then by construction, and by Axiom 3 in particular, D is computable by some Turing machine D_e , we make this more constructively explicit in the following Section 5.

Now D is stably sound by Lemma 3.20 and the assumption that F is sound. So by Lemma 4.3:

$$\neg(D_e \text{ is } D\text{-decided}).$$

In particular, $s(D_e)$ is not $Dec_{\mathcal{A}}(C(M_e))$ -decided, and so $s(D_e)$ is not $C(M_e)$ -stable (Lemma 3.20), i.e. $M \not\models s(D_e)$.

On the other hand,

$$\neg(D_e \text{ is } D\text{-decided}) \models \neg(D_e \text{ is } D_e\text{-decided}),$$

by definition. And so since $D_e \in \mathcal{D}_{\mathcal{T}}$ by construction, $s(D_e)$ is satisfied. Set $\alpha(M) := s(D_e)$ and we are done. $\square$

5. SYNTACTIC INCOMPLETENESS FOR STABLY COMPUTABLE THEORIES

Let $s : \mathcal{T} \rightarrow \mathcal{A}, T \mapsto s(T)$ be as in the previous section. Define

$$H : \mathcal{T} \times \mathcal{T} \times \mathbb{N} \rightarrow \{\pm\},$$

by $H(K, T, n) := (Dec_{\mathcal{A}}(C(Spec(K))))'(s(T), n)$. We can express H as the composition of the sequence of maps:

$$(5.1) \quad \mathcal{T} \times \mathcal{T} \times \mathbb{N} \xrightarrow{(Dec_{\mathcal{A}} \circ C \circ Spec) \times s \times id} \mathcal{T} \times \mathcal{A} \times \mathbb{N} \xrightarrow{id \times e_{\mathcal{A} \times \mathbb{N}}} \mathcal{T} \times \mathbb{N} \xrightarrow{U} \mathbb{N} \xrightarrow{e_{\{\pm\}}^{-1}} \{\pm\},$$

where the last map is:

$$\Sigma \mapsto \begin{cases} \text{undefined,} & \text{if } \Sigma \notin \{\pm\}_e \\ e_{\{\pm\}}^{-1}(\Sigma), & \text{otherwise.} \end{cases}$$

Thus, H is a composition of maps that are computable by the axioms of $\mathcal{S}$ and so H is computable. Hence, by Axiom 6 there is an associated total computable map:

$$(5.2) \quad Tur : \mathcal{T} \rightarrow \mathcal{T},$$

s.t. for each $K \in \mathcal{T}$, $Tur(K)$ computes the map

$$D^K : \mathcal{T} \times \mathbb{N} \rightarrow \{\pm\}, D^K(T, n) = H(K, T, n).$$

In what follows, $T \in \mathcal{T}_{\mathcal{A}}$, and we rename T' as M so $M : \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$. As usual, notation of the form $M \vdash \alpha$ means $M^s \vdash \alpha$.

Proposition 5.3. *For (M, T) as above:*

$$M^s \text{ is 1-consistent} \implies M \not\vdash s(\text{Tur}(T)).$$

$$M^s \text{ is 2-consistent} \implies M \not\vdash \neg s(\text{Tur}(T)).$$

Moreover, the sentence:

$$M^s \text{ is 1-consistent} \implies s(\text{Tur}(T))$$

is a theorem of PA under standard interpretation of all terms, (this will be further formalized in the course of the proof).

Proof. This proposition is meant to just be a theorem of set theory ZFC , however we of course avoid complete set theoretic formalization, as is common. Arithmetic is interpreted in set theory the standard way, using the standard set $\mathbb{N}$ of natural numbers. So for example, for $M : \mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$ a sentence of the form $M \vdash \alpha$ is a priori interpreted as a sentence of ZFC , however if M is a Turing machine this also can be interpreted as a sentence of PA , once Gödel encodings (the category $\mathcal{S}$) are invoked.

Set $N := (C \circ \text{Spec}(T))'$, in particular this is a speculative map $\mathbb{N} \rightarrow \mathcal{A} \times \{\pm\}$. Suppose that $M \vdash s(\text{Tur}(T))$. In particular, $s(\text{Tur}(T))$ is N -stable, and so by Lemma 3.20 $s(\text{Tur}(T))$ is $\text{Dec}_{\mathcal{A}}(C(\text{Spec}(T)))$ -decided, and so $\text{Tur}(T)$ is D^T -decided by definition. More explicitly, since D^T is total by construction, we deduce the sentence $\eta(T)$:

$$\exists m(\forall n \geq m) D(\text{Tur}(T), n) = +.$$

In other words:

$$(5.4) \quad (M \vdash s) \implies \eta(T)$$

is a theorem of ZFC .

If we translate $\eta(T)$ to an arithmetic sentence with the same name, then it can be chosen to have the form:

$$\exists m \forall n \gamma(m, n),$$

where $\gamma(m, n)$ is Q -decidable. The sentence $s(\text{Tur}(T))$ is assumed to be of the form $\beta(T) \wedge \neg \eta(T)$, where $\beta(T)$ is the arithmetic sentence equivalent in ZFC to $\text{Tur}(T) \in \mathcal{D}_{\mathcal{T}}$. Clearly, the translation maps $\mathcal{T} \rightarrow \mathcal{A}$, $T \mapsto \beta(T)$, $T \mapsto \eta(T)$ can be taken to be total and computable. Moreover, applying Lemma 3.18 (interpreted as a Theorem of PA): we get

$$(5.5) \quad PA \vdash \forall T \in \mathcal{T} \beta(T).$$

And so

$$(5.6) \quad PA \vdash (\eta(T) \implies \neg s(\text{Tur}(T))).$$

Moreover, ZFC proves:

$$\begin{aligned} \eta(T) &\implies \exists m \forall n Q \vdash \gamma(m, n), \quad \text{trivially} \\ &\implies \exists m : N \vdash \forall n \gamma(m, n), \quad \text{since } N \text{ is speculative} \\ &\implies N \vdash \eta(T), \quad (\text{existential introduction}) \\ &\implies N \vdash \neg s(\text{Tur}(T)), \quad \text{by (5.6) and since } N^s \supset PA. \end{aligned}$$

And so combining with (5.4), (5.6) ZFC proves:

$$(M \vdash s) \implies (N \vdash s) \wedge (N \vdash \neg s).$$

Since by Theorem 3.28

$$M^s \text{ is 1-consistent} \implies N^s \text{ is consistent},$$

it follows:

$$(5.7) \quad ZFC \vdash (M^s \text{ is 1-consistent} \implies M \not\vdash s(Tur(T)) \quad (\text{moreover } N \not\vdash s(Tur(T)))).$$

Now suppose

$$(M^s \text{ is 2-consistent}) \wedge (M \vdash \neg s(Tur(T))).$$

Since we have (5.5), and since $M \vdash PA$ it follows that $M \vdash \eta(T)$.

Set

$$\phi(m) = \forall n \gamma(m, n).$$

Now,

$$\begin{aligned} M \vdash \eta(T) &\iff M \vdash \exists m \phi(m) \\ &\implies \exists m M \not\vdash \neg \phi(m), \quad \text{by 2-consistency.} \\ &\implies \exists m \forall n Q \vdash \gamma(m, n), \quad \text{as } M^s \supset Q \text{ and } \gamma(m, n) \text{ is } Q\text{-decidable.} \end{aligned}$$

In other words, ZFC proves:

$$(M^s \text{ is 2-consistent} \wedge (M \vdash \neg s)) \implies \eta(T).$$

And ZFC proves:

$$\eta(T) \implies N \vdash s(Tur(T)),$$

by definitions. So ZFC proves:

$$\begin{aligned} (M^s \text{ is 2-consistent} \wedge (M \vdash \neg s(Tur(T)))) &\implies N \vdash s(Tur(T)) \\ &\implies N^s \text{ is inconsistent} \\ &\implies M^s \text{ is not 1-consistent, by Theorem 3.28} \\ &\implies M^s \text{ is not 2-consistent.} \end{aligned}$$

And so ZFC proves:

$$M^s \text{ is 2-consistent} \implies M \not\vdash \neg s(Tur(T)).$$

Now for the last part of the proposition. We essentially just further formalize (5.7) and its consequences in PA . In what follows by equivalence of sentences we mean equivalence in ZFC . The correspondence of sentences under equivalence is the standard kind of correspondence assigning predicates involving Turing machines predicates in PA . The basic form of such correspondences is already constructed by Turing [1], so that we will not elaborate. In particular, the correspondences are computable, which just means that the corresponding maps $\mathcal{T} \rightarrow \mathcal{A}$ are computable.

Definition 5.8. We say that $T \in \mathcal{T}$ is **stably 1-consistent** if $T \in \mathcal{T}_A$ and T^s is 1-consistent, (Notation 2). The sentence “ T^s is 1-consistent”, can specifically be taken to be the arithmetic sentence $Con_{\sigma,1}$ for σ the natural Σ_2 definition of T^s and $Con_{\sigma,1}$ the consistency sentence as in [8, Section 5].

Then the sentence:

$$T \text{ is stably 1-consistent}$$

is equivalent to an arithmetic sentence we denote:

$$1 - con^s(T).$$

The sentence $Spec(T) \not\vdash s(Tur(T))$ is equivalent to an arithmetic sentence we call:

$$\omega(T).$$

By the proof of the first part of the proposition, that is by (5.7),

$$(5.9) \quad ZFC \vdash \forall T \in \mathcal{T} (1 - con^s(T) \implies \omega(T)).$$

But we also have:

$$(5.10) \quad PA \vdash \forall T \in \mathcal{T} (1 - \text{con}^s(T) \implies \omega(T)),$$

since the first part of the proposition can be formalized in PA , in fact the only interesting theorems we used are Lemma 3.20, and Theorem 3.28 which are obviously theorems of PA .

Now, by Lemma 3.20 and the construction of H :

$$PA \vdash \forall T \in \mathcal{T} \omega(T) \iff \neg \eta(T).$$

So:

$$PA \vdash \forall T \in \mathcal{T} (\beta(T) \wedge \omega(T) \iff s(\text{Tur}(T))),$$

Combining with (5.5) and with (5.10) we get:

$$PA \vdash \forall T \in \mathcal{T} (1 - \text{con}^s(T) \implies s(\text{Tur}(T))).$$

So for (M, T) as in the statement, if we formally interpret the sentence “ M^s is 1-consistent” as the arithmetic sentence $1 - \text{con}^s(T)$, then this formalizes and proves the last part of the proposition. $\square$

Proof of Theorem 1.5. The computable map $\mathcal{G}$ is defined to be $T \mapsto s(\text{Tur}(T))$. Strictly, speaking for this to be a computable map to $\mathcal{L}$ we need that the interpretation of $\mathcal{A}$ in $\mathcal{L}$ is suitably computable, so that $s(\text{Tur}(T))$, determines a sentence of $\mathcal{L}$. For example, this is the case if $\mathcal{L}$ is the language of ZFC and we take standard interpretations. Then the theorem follow by the proposition above, applied to the pair $((K_{\mathcal{A}}(T))', K_{\mathcal{A}}(T))$. $\square$

Proof of Corollary 1.7. Suppose that

$$F \vdash F \text{ is 1-consistent} \wedge F \vdash T \text{ computes } F \wedge F \text{ is 1-consistent}$$

then since $F \vdash PA$,

$$(F \vdash \mathcal{G}(T)) \wedge F \text{ is 1-consistent}$$

by Part 3 of Theorem 1.5. But this contradicts Part 1 of Theorem 1.5. $\square$

In what follows as previously $\mathcal{Z}$ denotes the first order language of set theory.

Definition 5.11. Suppose we are given a theory F in some language $\mathcal{L}$, such that $F \vdash^i ZFC$. Then we say that it is **strongly consistent** if there is a standard model $\mathcal{U}$ for $F_{\mathcal{Z}}$. More specifically, $\mathcal{U} = V_{\kappa}$ for κ a strongly inaccessible cardinal and V_{κ} the corresponding stage in the Von Neumann hierarchy. And so that the interpretation of $\in$ in $\mathcal{U}$ is the natural one.

In particular, strong consistency implies the set theoretic sentence:

$$(\forall \alpha \in \overline{F}) \alpha^{\mathcal{U}},$$

where $\alpha^{\mathcal{U}}$ denotes the sentence α , with domain of discourse restricted to $\mathcal{U}$, that is any variables range over $\mathcal{U}$, and where $\overline{F}$ denotes the logical closure of F .

Proof of Theorem 1.8. Let $\mathcal{U}$ be a standard model of F as above. Suppose $F \vdash F$ is 1-consistent, and also

$$F \vdash \exists T T \text{ stably computes } F.$$

Then since we have

$$(5.12) \quad (\forall \alpha \in \overline{F}) \alpha^{\mathcal{U}},$$

we obtain:

$$(F \text{ is 1-consistent}) \wedge (\exists T T \text{ stably computes } F) \wedge (F \vdash F \text{ is 1-consistent}),$$

that is

$$\exists T (((T')^s \text{ is 1-consistent}) \wedge ((T')^s \vdash (T')^s \text{ is 1-consistent})).$$

Rephrasing Corollary 1.7 we get:

$$ZFC \vdash \forall T : ((T')^s \text{ is 1-consistent}) \implies (T')^s \not\vdash (T')^s \text{ is 1-consistent}.$$

And $F \vdash ZFC$ and so again by (5.12) we get:

$$\neg((\exists T ((T')^s \text{ is 1-consistent}) \wedge ((T')^s \vdash (T')^s \text{ is 1-consistent})).$$

So we have obtained a contradiction (in ZFC). $\square$

APPENDIX A. STABLE COMPUTABILITY AND PHYSICS - GÖDEL'S DISJUNCTION AND PENROSE

We now give some partly physical motivation for the theory above, and in particular for the notion of stable computability. As this work is aimed at mathematicians, we aim to be very brief. But developing this appendix would be very interesting in an appropriate venue.

We may say that a physical process is *absolutely not Turing computable*, if it is not Turing computable in any “sufficiently physically accurate” mathematical model. For example, it is well known (see for instance [4]) that solutions of fluid flow and N -body problems are generally non Turing computable (over $\mathbb{Z}$, and probably over $\mathbb{R}$ cf. [3]) as modeled in mathematics of classical mechanics. But in a more physically accurate and fundamental model both of the processes above may become computable.

The question posed by Turing [2], but also by Gödel [7, 310] and more recently and much more expansively by Penrose [11], [12], [13] is:

Question 1. Are there absolutely not Turing computable physical processes? And moreover, are brain processes absolutely not Turing computable?

A.0.1. *Gödel's disjunction.* Gödel argued for a 'yes' answer to Question 1, see [7, pg. 310], relating the question to existence of absolutely unsolvable Diophantine problems, see also Feferman [6], and Koellner [9], [10] for a discussion.

We now discuss the question from the perspective of our main results. First by an idealized mathematician, we mean a mapping $H : \mathbb{N} \rightarrow \mathcal{L} \times \{\pm\}$, with $\mathcal{L}$ a first order language. The language $\mathcal{L}$ is meant to be chosen so that it is sufficient to formalize physical laws (we don't need to formalize everything, the context will be self apparent). This might be the language of set theory, but perhaps one needs more.⁴ The mapping H is meant to be the actual time stamped output of a mathematician, idealized so that their brain does not deteriorate in time, see also Remark 1.2.

We set $\mathcal{H} = H^s$. Soundness and in particular strong consistency (Definition 5.11) of the stabilization $\mathcal{H}$ is not an unreasonable hypothesis for our “idealized” mathematician, as mathematical knowledge does appear to stabilize on truth. Here we are using ‘stabilize’ in the more common language sense, but in this setting this is equivalent to the mathematical stabilization H^s .

Without delving deeply into interpretations, we suppose the following axioms for $\mathcal{H}$.

- (1) $\mathcal{H}$ is $\mathcal{L}$ -definable. Meaning, that it is definable with respect to some $\mathcal{L}$ -structure (informally, this is obtained from some mathematical (sub)model of the physical universe).⁵
- (2) $\mathcal{H} \vdash^i ZFC$, recall the Definition 2.5. (And so in particular, $\mathcal{H} \vdash^i Q$, Corollary 2.7.)
- (3) The “Penrose property”: $\mathcal{H}_{\mathcal{Z}} \vdash \mathcal{H}$ is 1-consistent, where $\mathcal{H}_{\mathcal{Z}}$ is as in Theorem 1.8.

⁴It is of course possible that no such language exists, but this is not our expectation: mathematics is unreasonably effective at formalizing the universe.

⁵This is natural: as H is determined by some physical processes, it can then be defined with respect to $\mathcal{L}$, since by assumption the physical laws are formalizable in an $\mathcal{L}$ -theory.

The ‘Penrose property’ is of course motivated by ideas of Roger Penrose as appearing in references above. This property makes sense if our idealized mathematician knows the definition of $\mathcal{H}$, and asserts his/her soundness, and hence 1-consistency of $\mathcal{H}_A$ in particular. The possibility that our mathematician indeed knows the definition of $\mathcal{H}$ is perhaps not unlikely. Just map the brain, its synapses, etc.; then assuming one knows the working of all underlying physical processes, use this to reconstruct the set theoretic definition of $\mathcal{H}$. This would be a fantastically difficult thing to do, but not a priori impossible. Given this, there is no obvious reason to reject the above axioms.

Applying Theorem 1.8 we then get the following pseudo-theorem. (It is not a ‘theorem’ since of course $\mathcal{H}$ at the moment does not have a mathematical definition.)

Pseudo-theorem A.1. *One of the following holds:*

- (1) $\mathcal{H}$ is not strongly consistent.
- (2) $\mathcal{H}_Z \not\vdash \mathcal{H}$ is stably c.e.. (In particular, our idealized mathematician, represented by $\mathcal{H}$, is unable to disprove existence of absolutely non Turing computable physical processes.)

Acknowledgements. Peter Koellner for helpful discussions on related topics.

REFERENCES

- [1] A.M. TURING, *On computable numbers, with an application to the entscheidungsproblem*, Proceedings of the London mathematical society, s2-42 (1937).
- [2] ———, *Computing machines and intelligence*, Mind, 49 (1950), pp. 433–460.
- [3] L. BLUM, M. SHUB, AND S. SMALE, *On a theory of computation and complexity over the real numbers: NP-completeness, recursive functions and universal machines.*, Bull. Am. Math. Soc., New Ser., 21 (1989), pp. 1–46.
- [4] E. P.-S. D. P. F. CARDONA, ROBERT; MIRANDA, *Constructing turing complete euler flows in dimension 3*, PNAS, 118 (2021).
- [5] S. FEFERMAN, *1-consistency and faithful interpretations*, 6 (1962).
- [6] ———, *Are There Absolutely Unsolvable Problems? Gödel’s Dichotomy*, Philosophia Mathematica, 14 (2006), pp. 134–152.
- [7] K. GÖDEL, *Collected Works III* (ed. S. Feferman), New York: Oxford University Press, 1995.
- [8] M. KIKUCHI AND T. KURAHASHI, *Generalizations of Gödel’s incompleteness theorems for Σ_n -definable theories of arithmetic*, Rev. Symb. Log., 10 (2017), pp. 603–616.
- [9] P. KOELLNER, *On the Question of Whether the Mind Can Be Mechanized, I: From Gödel to Penrose*, Journal of Philosophy, 115 (2018), pp. 337–360.
- [10] ———, *On the question of whether the mind can be mechanized, ii: Penrose’s new argument*, Journal of Philosophy, 115 (2018), pp. 453–484.
- [11] R. PENROSE, *Shadows of the mind*, 1994.
- [12] ———, *Beyond the shadow of a doubt*, Psyche, (1996).
- [13] ———, *Road to Reality*, 2004.
- [14] R. I. SOARE, *Turing computability. Theory and applications*, Berlin: Springer, 2016.

UNIVERSITY OF COLIMA, DEPARTMENT OF SCIENCES, CUICBAS

Email address: yasha.savelyev@gmail.com